BookBridge

Implementierung der Datenbank

PostgreSQL 17.11

Ausgeführte Tabellen, Testdaten und fachliche Abläufe

DLBDSPBDM01 / Arbeitsstand 18. Oktober 2026

Ausgeführte Implementierung

Der Entwurf aus Phase 1 liegt als lauffähiges PostgreSQL-Schema vor.

Prüfgegenstand	Tatsächlicher Stand
Basistabellen und Attribute	18 Tabellen mit 142 Spalten
Referenzielle Integrität	32 Fremdschlüssel
Sinnvoll belegte Testdaten	1.431 Zeilen, mindestens 12 je Tabelle
Ausgeführte Integrationstests	23 bestanden, 0 fehlgeschlagen
Betrieb und Zeitzone	PostgreSQL 17.11, Europe/Berlin

Eine unabhängige Neuinstallation bestätigt die veröffentlichten SQL-Dateien.

Testdaten je Entität

Alle 18 Tabellen enthalten mehr als die geforderten zehn Datensätze.

Tabelle	Zeilen
users	24
addresses	24
publishers	12
genres	12
languages	12
authors	24
books	24
book_authors	27
book_copies	24

Tabelle	Zeilen
pickup_slots	48
loans	48
loan_proposals	24
loan_extensions	32
payments	24
shipments	34
notifications	786
generated_documents	204
reviews	48

Gesamt: 1.431 Zeilen

Benutzer: users

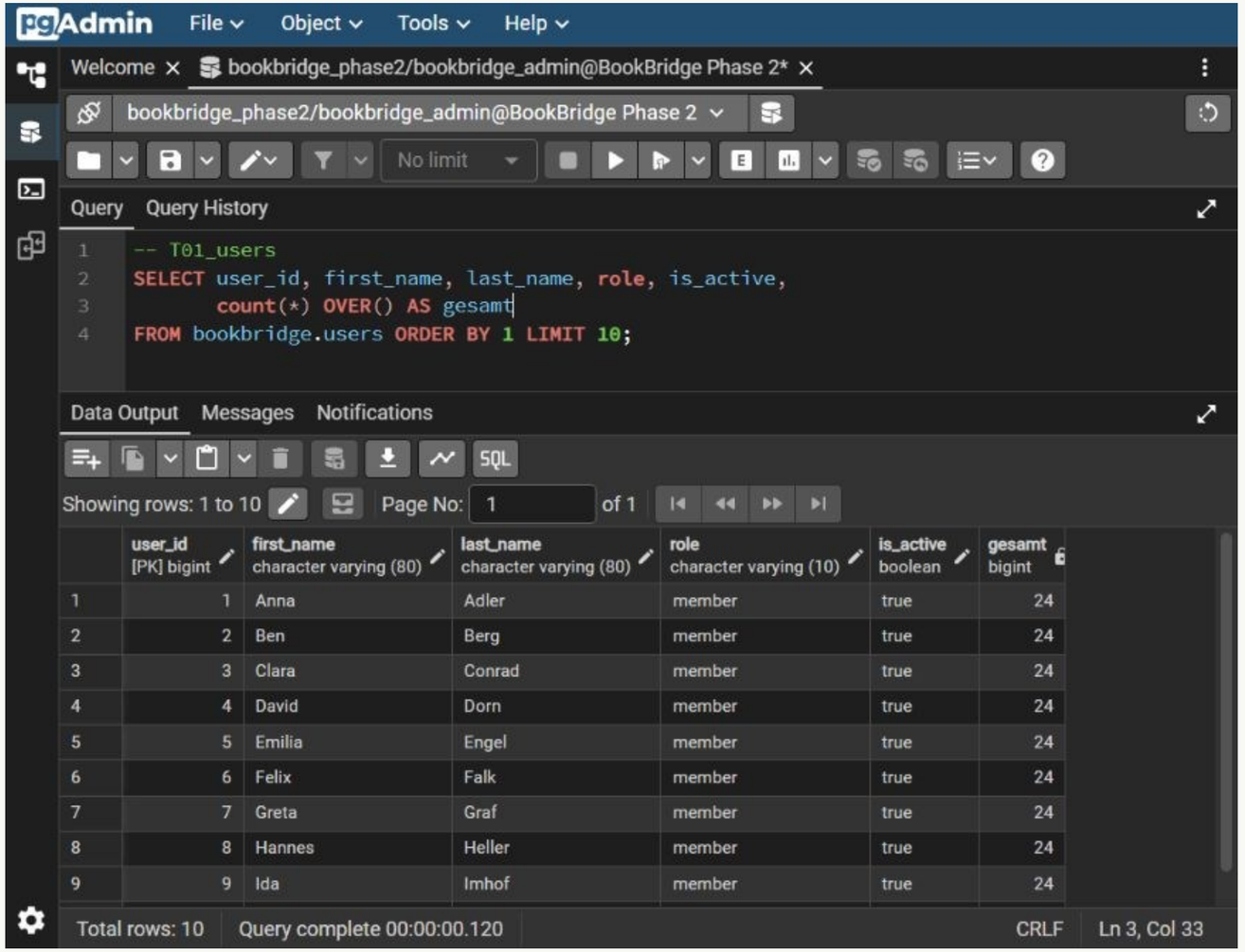
Mitgliederkonten und Administratoren

CREATE TABLE / SQL-Auszug

```
CREATE TABLE bookbridge.users (
    user_id BIGINT GENERATED BY DEFAULT AS
  IDENTITY NOT NULL,
    email VARCHAR(254) NOT NULL,
    role VARCHAR(10) NOT NULL DEFAULT
  'member',
    is_active BOOLEAN NOT NULL DEFAULT
  true,
    -- Weitere Spalten im SQL-Skript.
    CONSTRAINT users_pk PRIMARY KEY
  (user_id)
);
```

E-Mail-Adressen bleiben ohne Beachtung der Großschreibung eindeutig. Die Rolle ist member oder admin.

8 Spalten / 24 Datensätze



Anschriften: addresses

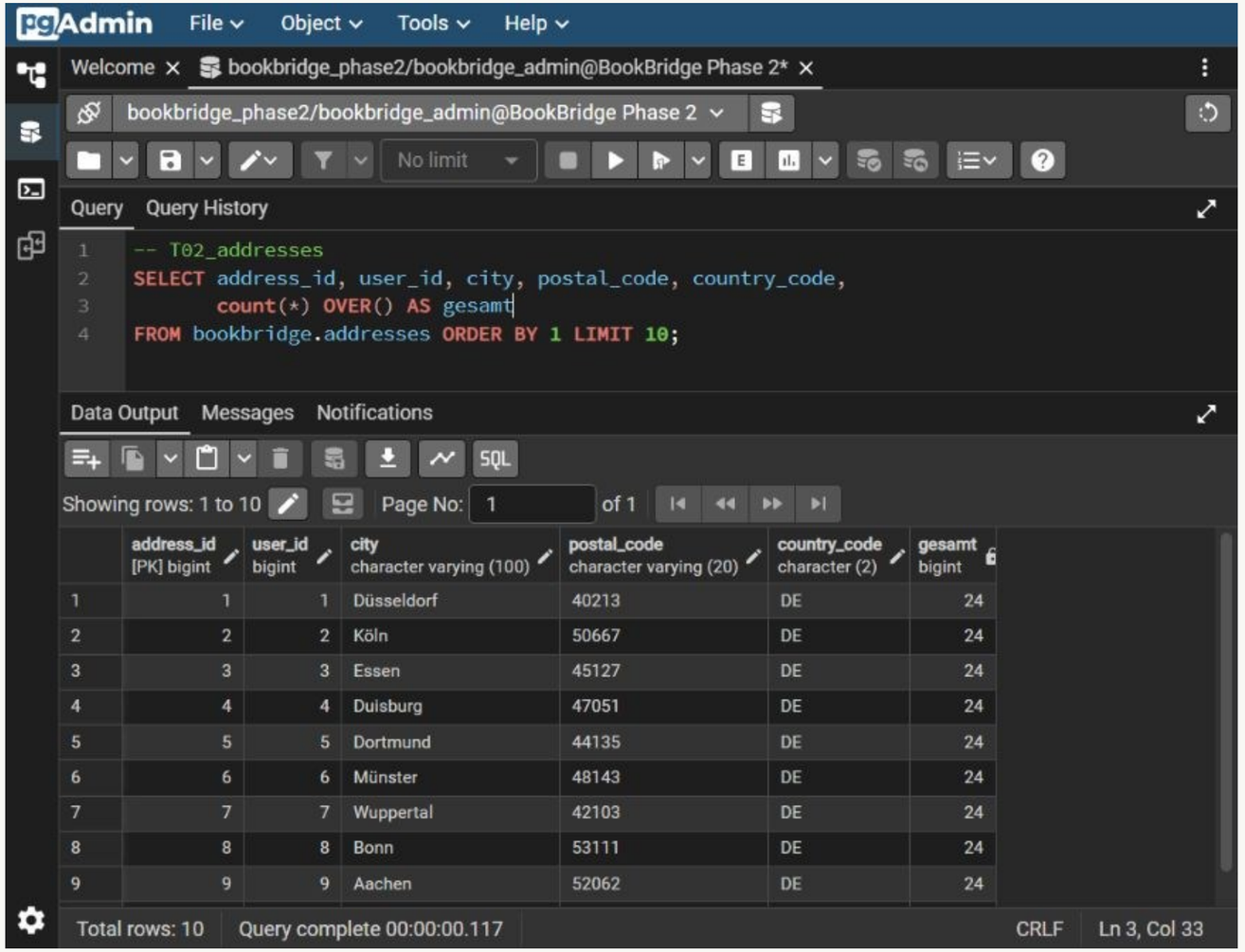
Private Anschriften und optionale WGS84-Koordinaten

CREATE TABLE / SQL-Auszug

```
CREATE TABLE bookbridge.addresses (
    address_id BIGINT GENERATED BY DEFAULT
  AS IDENTITY NOT NULL,
    user_id BIGINT NOT NULL,
    latitude NUMERIC(9,6),
    longitude NUMERIC(9,6),
    -- Weitere Spalten im SQL-Skript.
    CONSTRAINT addresses_pk PRIMARY KEY
  (address_id)
);
```

Breiten- und Längengrad stehen gemeinsam fest oder fehlen gemeinsam. Die Anschrift gehört genau einem Mitglied.

9 Spalten / 24 Datensätze



Verlage: publishers

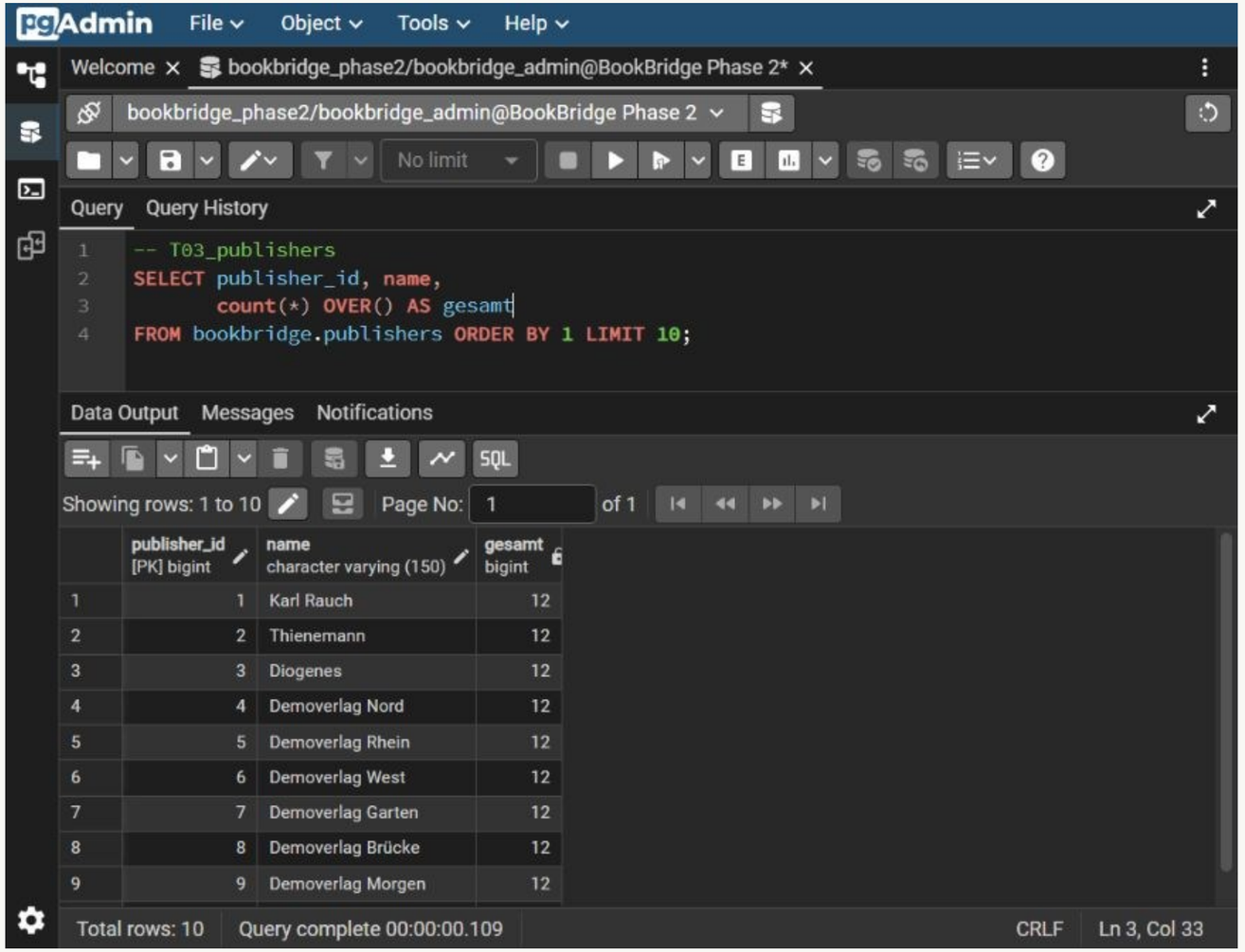
Verlage bibliografischer Ausgaben

CREATE TABLE / SQL-Auszug

```
CREATE TABLE bookbridge.publishers (
    publisher_id BIGINT GENERATED BY DEFAULT
  AS IDENTITY NOT NULL,
    name VARCHAR(150) NOT NULL,
    -- Weitere Constraints im SQL-Skript.
    CONSTRAINT publishers_pk PRIMARY KEY
  (publisher_id)
);
```

Der Verlagsname ist eindeutig. Eine Ausgabe darf bei unbekanntem Verlag ohne Zuordnung bleiben.

2 Spalten / 12 Datensätze



Genres: genres

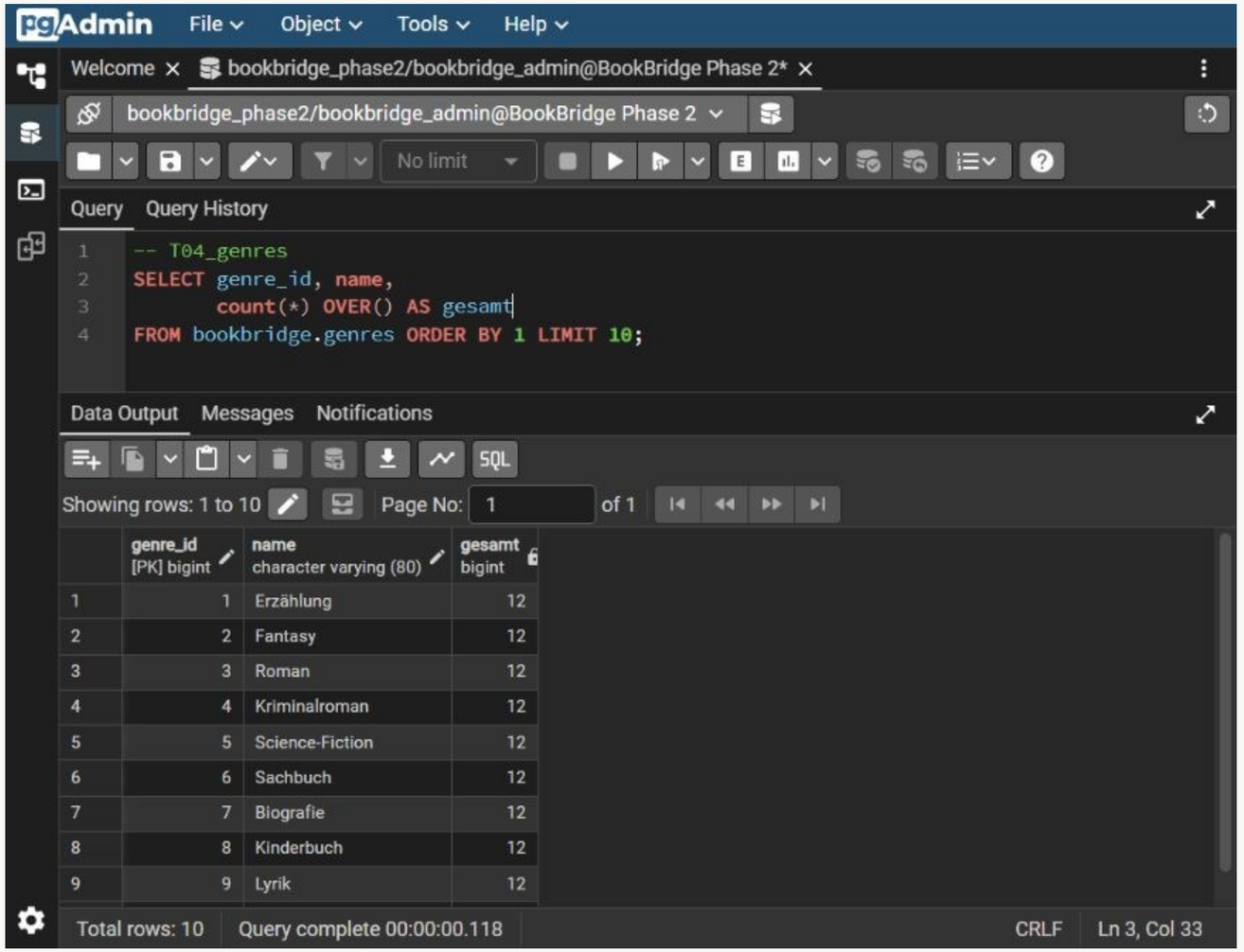
Primäre Genres des Katalogs

CREATE TABLE / SQL-Auszug

```
CREATE TABLE bookbridge.genres (
    genre_id BIGINT GENERATED BY DEFAULT AS
  IDENTITY NOT NULL,
    name VARCHAR(80) NOT NULL,
    -- Weitere Constraints im SQL-Skript.
    CONSTRAINT genres_pk PRIMARY KEY
  (genre_id)
);
```

Ein Genre hat einen eindeutigen, nicht leeren Namen.
Jede Ausgabe verweist auf ein primäres Genre.

2 Spalten / 12 Datensätze



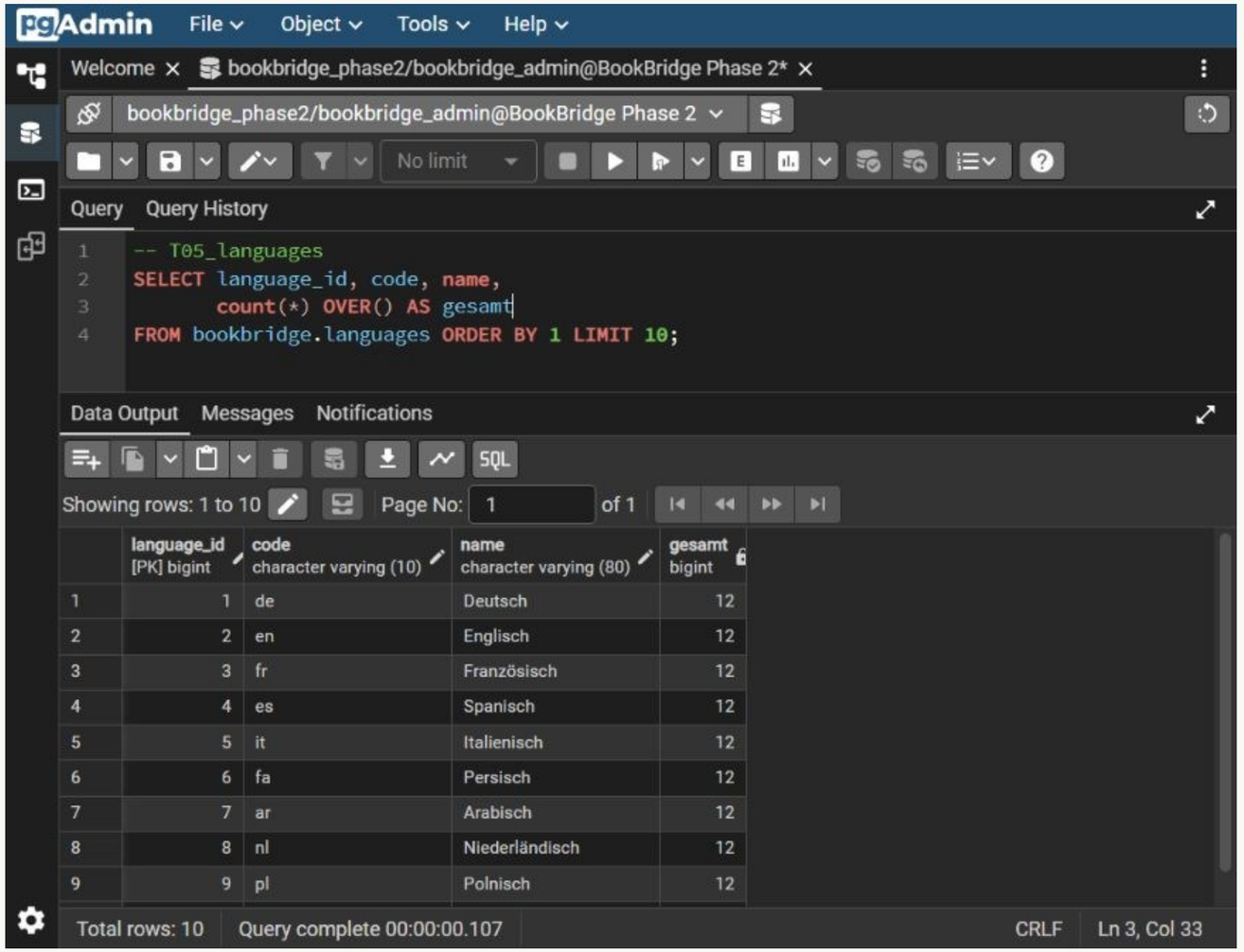
Sprachen: languages

Sprachen der bibliografischen Ausgaben

CREATE TABLE / SQL-Auszug

```
CREATE TABLE bookbridge.languages (
    language_id BIGINT GENERATED BY DEFAULT
  AS IDENTITY NOT NULL,
    code VARCHAR(10) NOT NULL,
    name VARCHAR(80) NOT NULL,
    -- Weitere Constraints im SQL-Skript.
    CONSTRAINT languages_pk PRIMARY KEY
  (language_id)
);
```

Der Sprachcode bleibt eindeutig. Jede Ausgabe hat genau eine primäre Sprache.



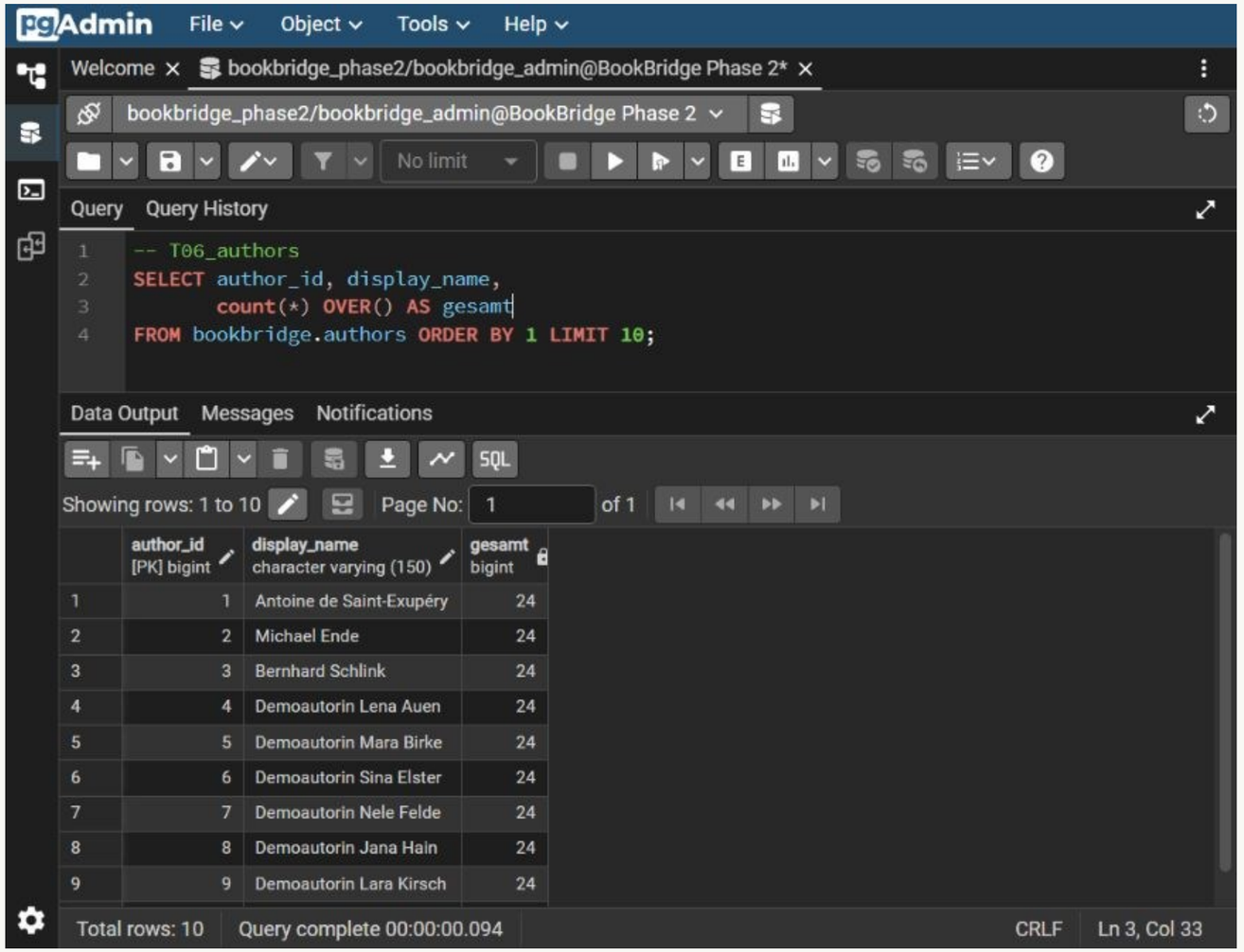
3 Spalten / 12 Datensätze

Autoren: authors

Personen und Namen hinter den Werken

CREATE TABLE / SQL-Auszug

```
CREATE TABLE bookbridge.authors (
    author_id BIGINT GENERATED BY DEFAULT AS
  IDENTITY NOT NULL,
    display_name VARCHAR(150) NOT NULL,
    -- Weitere Constraints im SQL-Skript.
    CONSTRAINT authors_pk PRIMARY KEY
  (author_id)
);
```



Autoren bestehen unabhängig von ihren Ausgaben. Die Zuordnung und ihre Reihenfolge stehen in book_authors.

2 Spalten / 24 Datensätze

Bibliografische Ausgaben: books

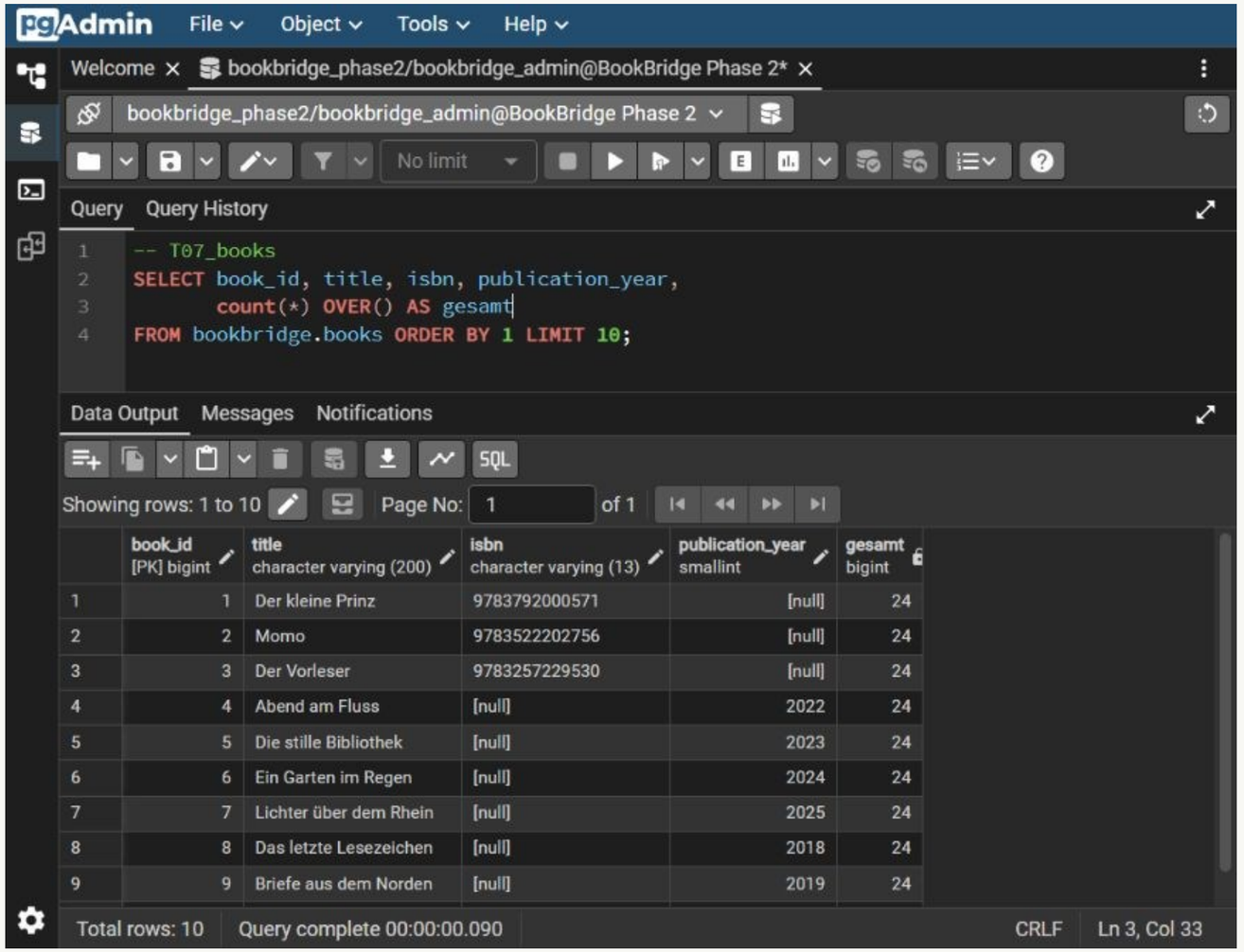
Bibliografische Ausgaben unabhängig vom Eigentümer

CREATE TABLE / SQL-Auszug

```
CREATE TABLE bookbridge.books (
    book_id BIGINT GENERATED BY DEFAULT AS
  IDENTITY NOT NULL,
    title VARCHAR(200) NOT NULL,
    isbn VARCHAR(13),
    genre_id BIGINT NOT NULL,
    -- Weitere Spalten im SQL-Skript.
    CONSTRAINT books_pk PRIMARY KEY
  (book_id)
);
```

ISBN-Prüfziffer und Eindeutigkeit gelten bei gesetzter ISBN. Spätestens zum Transaktionsende hat jede Ausgabe mindestens einen Autor.

10 Spalten / 24 Datensätze



Autorenzuordnung: book_authors

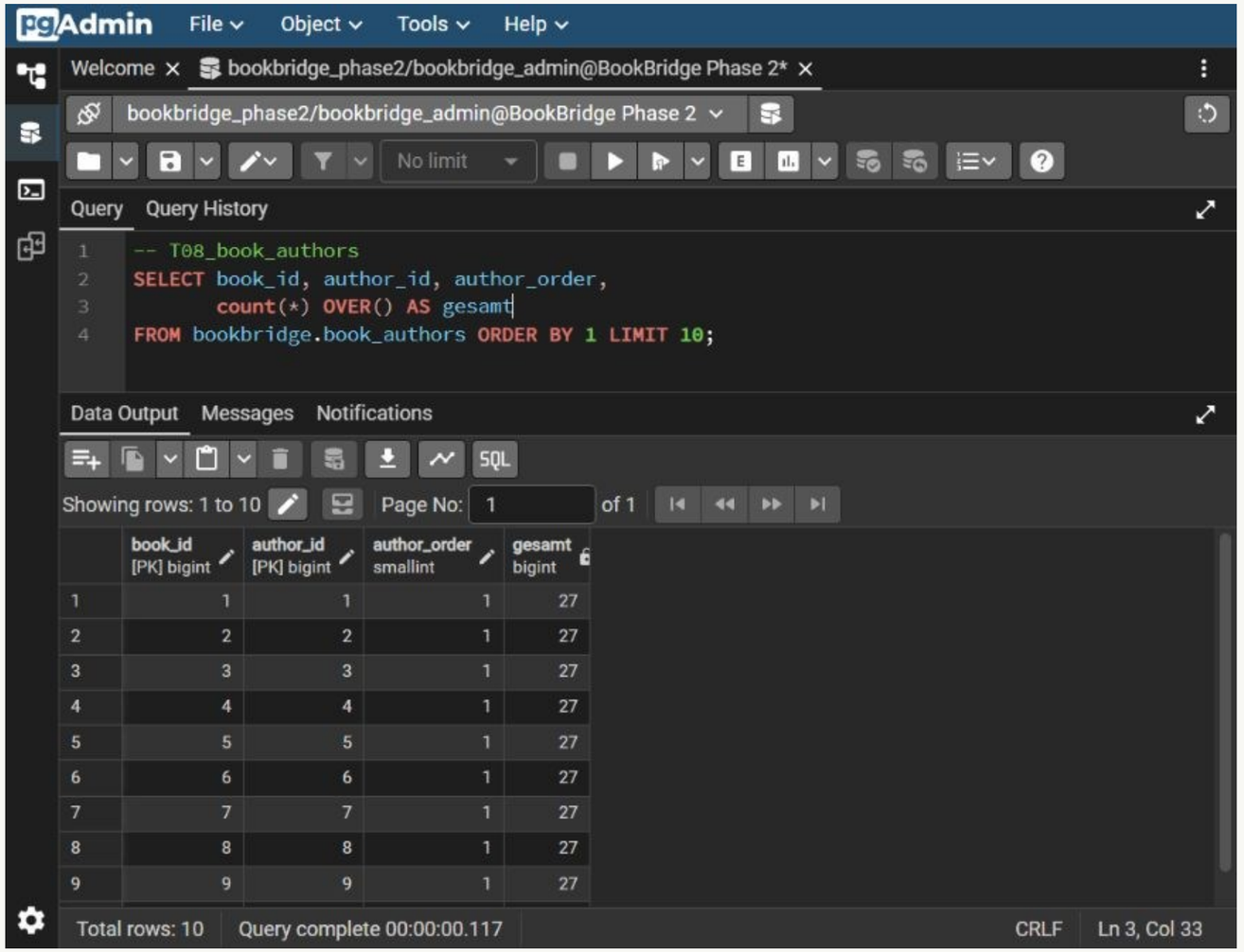
Autoren einer Ausgabe in festgelegter Reihenfolge

CREATE TABLE / SQL-Auszug

```
CREATE TABLE bookbridge.book_authors (
    book_id BIGINT NOT NULL,
    author_id BIGINT NOT NULL,
    author_order SMALLINT NOT NULL,
    -- Weitere Constraints im SQL-Skript.
    CONSTRAINT book_authors_pk PRIMARY KEY
  (book_id,author_id)
);
```

Der zusammengesetzte Primärschlüssel verhindert doppelte Autorenzuordnungen. Die Reihenfolge je Ausgabe bleibt eindeutig.

3 Spalten / 27 Datensätze



Physische Exemplare: book_copies

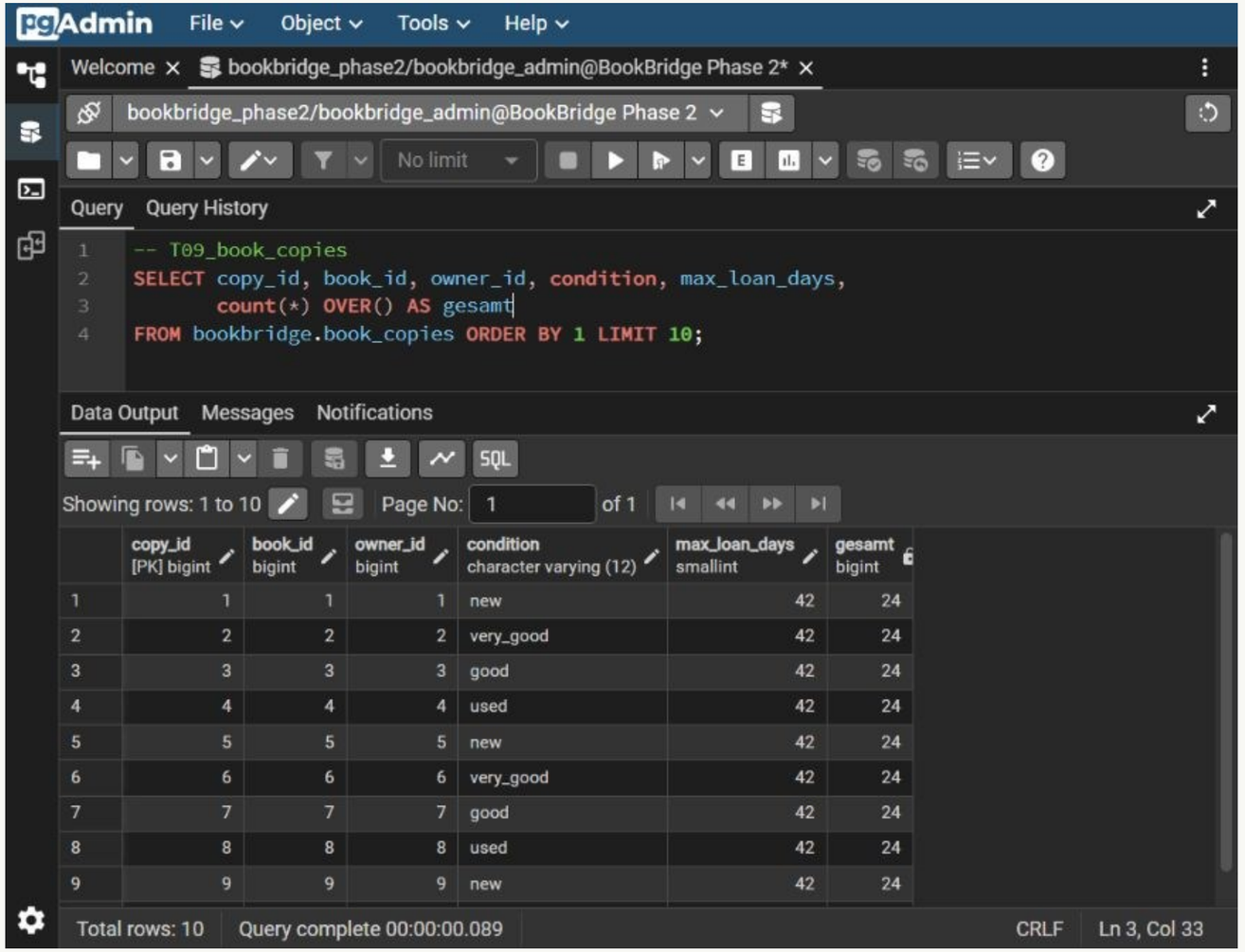
Angebot und Eigentum eines physischen Exemplars

CREATE TABLE / SQL-Auszug

```
CREATE TABLE bookbridge.book_copies (
    copy_id BIGINT GENERATED BY DEFAULT AS
  IDENTITY NOT NULL,
    book_id BIGINT NOT NULL,
    owner_id BIGINT NOT NULL,
    max_loan_days SMALLINT NOT NULL DEFAULT
  14,
    -- Weitere Spalten im SQL-Skript.
    CONSTRAINT book_copies_pk PRIMARY KEY
  (copy_id)
);
```

Angebotsdauer: 1 bis 90 Tage. Ein aktives Angebot benötigt eine passende Eigentümeranschrift und bei Abholung aktive Zeitfenster.

11 Spalten / 24 Datensätze



Abholzeitfenster: pickup_slots

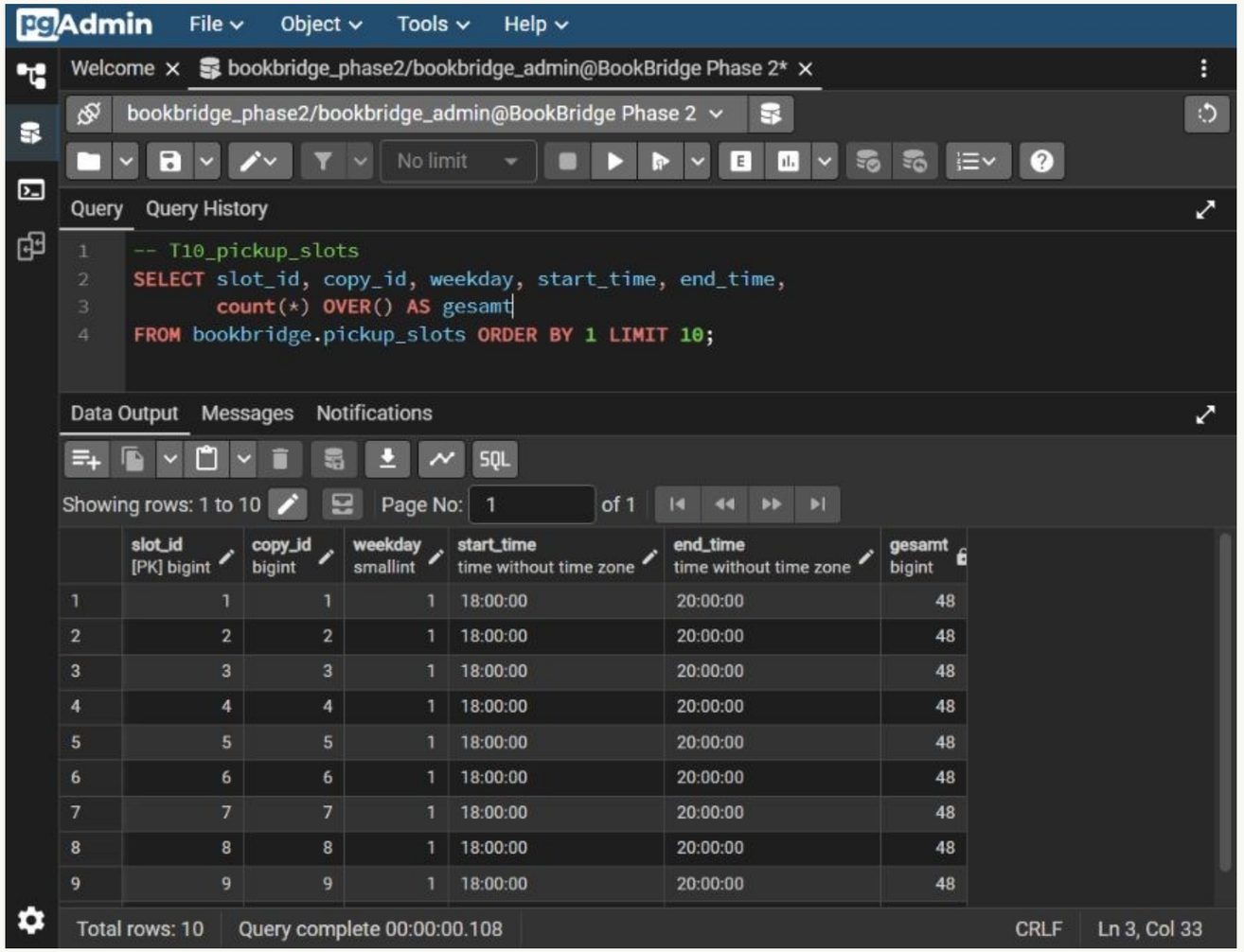
Wiederkehrende Wochenzeitfenster zur Abholung

CREATE TABLE / SQL-Auszug

```
CREATE TABLE bookbridge.pickup_slots (
    slot_id BIGINT GENERATED BY DEFAULT AS
  IDENTITY NOT NULL,
    copy_id BIGINT NOT NULL,
    weekday SMALLINT NOT NULL,
    start_time TIME NOT NULL,
    -- Weitere Spalten im SQL-Skript.
    CONSTRAINT pickup_slots_pk PRIMARY KEY
  (slot_id)
);
```

Wochenzeitfenster eines Exemplars dürfen sich nicht überschneiden. Der konkrete Termin steht später zusätzlich in loans.

6 Spalten / 48 Datensätze



Ausleihvorgänge: loans

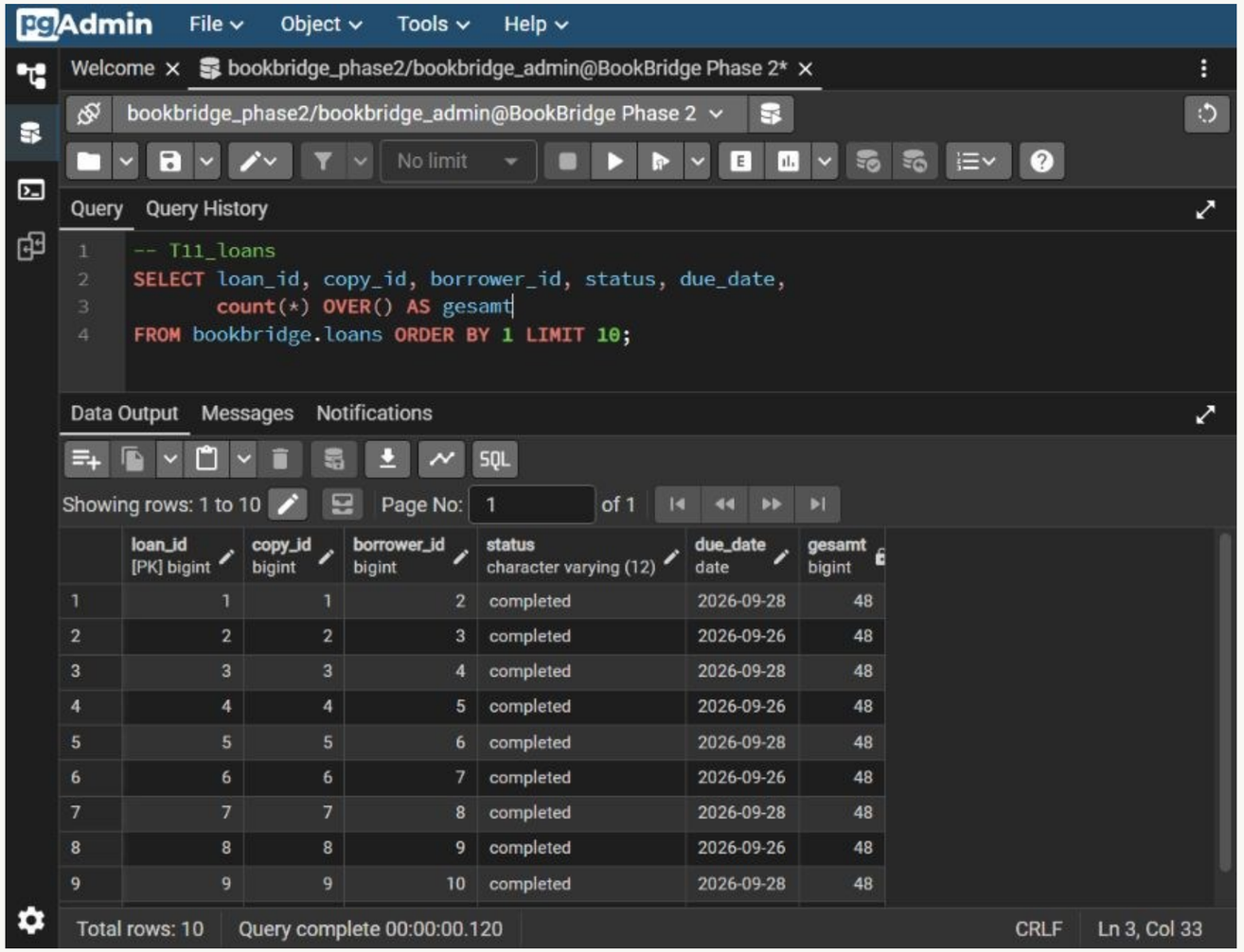
Anfrage, Vereinbarung, Empfang und Rückgabe

CREATE TABLE / SQL-Auszug

```
CREATE TABLE bookbridge.loans (
    loan_id BIGINT GENERATED BY DEFAULT AS
  IDENTITY NOT NULL,
    copy_id BIGINT NOT NULL,
    borrower_id BIGINT NOT NULL,
    status VARCHAR(12) NOT NULL DEFAULT
  'requested',
    -- Weitere Spalten im SQL-Skript.
    CONSTRAINT loans_pk PRIMARY KEY
  (loan_id)
);
```

Genau eine verbindliche Ausleihe je Exemplar. Abholung, Empfang und Rückgabe folgen der zulässigen Ereignisreihenfolge.

21 Spalten / 48 Datensätze



Gegenvorschläge: loan_proposals

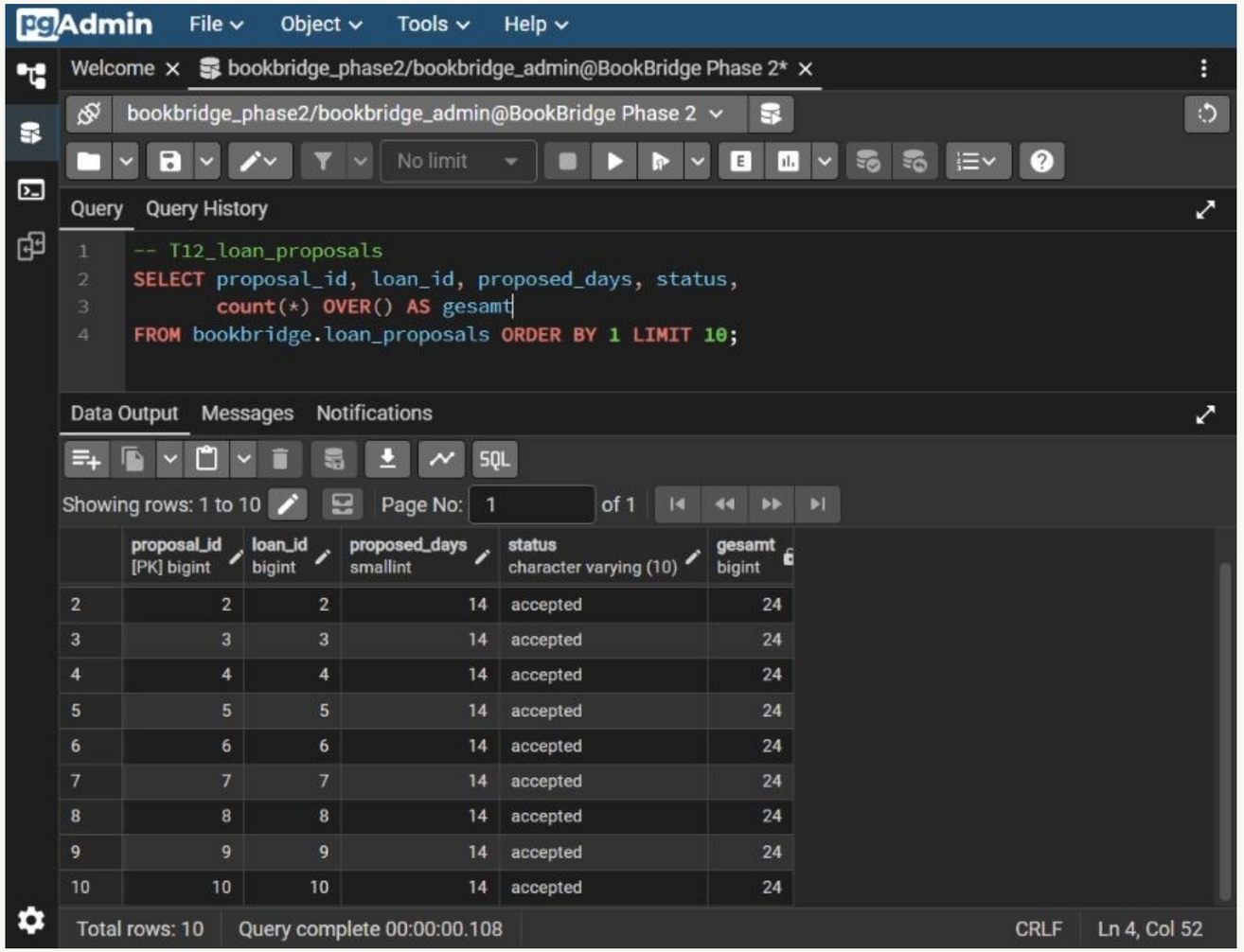
Gegenvorschläge zur anfänglichen Leihdauer

CREATE TABLE / SQL-Auszug

```
CREATE TABLE bookbridge.loan_proposals (
    proposal_id BIGINT GENERATED BY DEFAULT
  AS IDENTITY NOT NULL,
    loan_id BIGINT NOT NULL,
    proposed_days SMALLINT NOT NULL,
    status VARCHAR(10) NOT NULL DEFAULT
  'pending',
    -- Weitere Spalten im SQL-Skript.
    CONSTRAINT loan_proposals_pk PRIMARY KEY
  (proposal_id)
);
```

Pro Ausleihe bleibt höchstens ein Gegenvorschlag offen. Die zuständige andere Person nimmt an oder lehnt ab.

8 Spalten / 24 Datensätze



Verlängerungen: loan_extensions

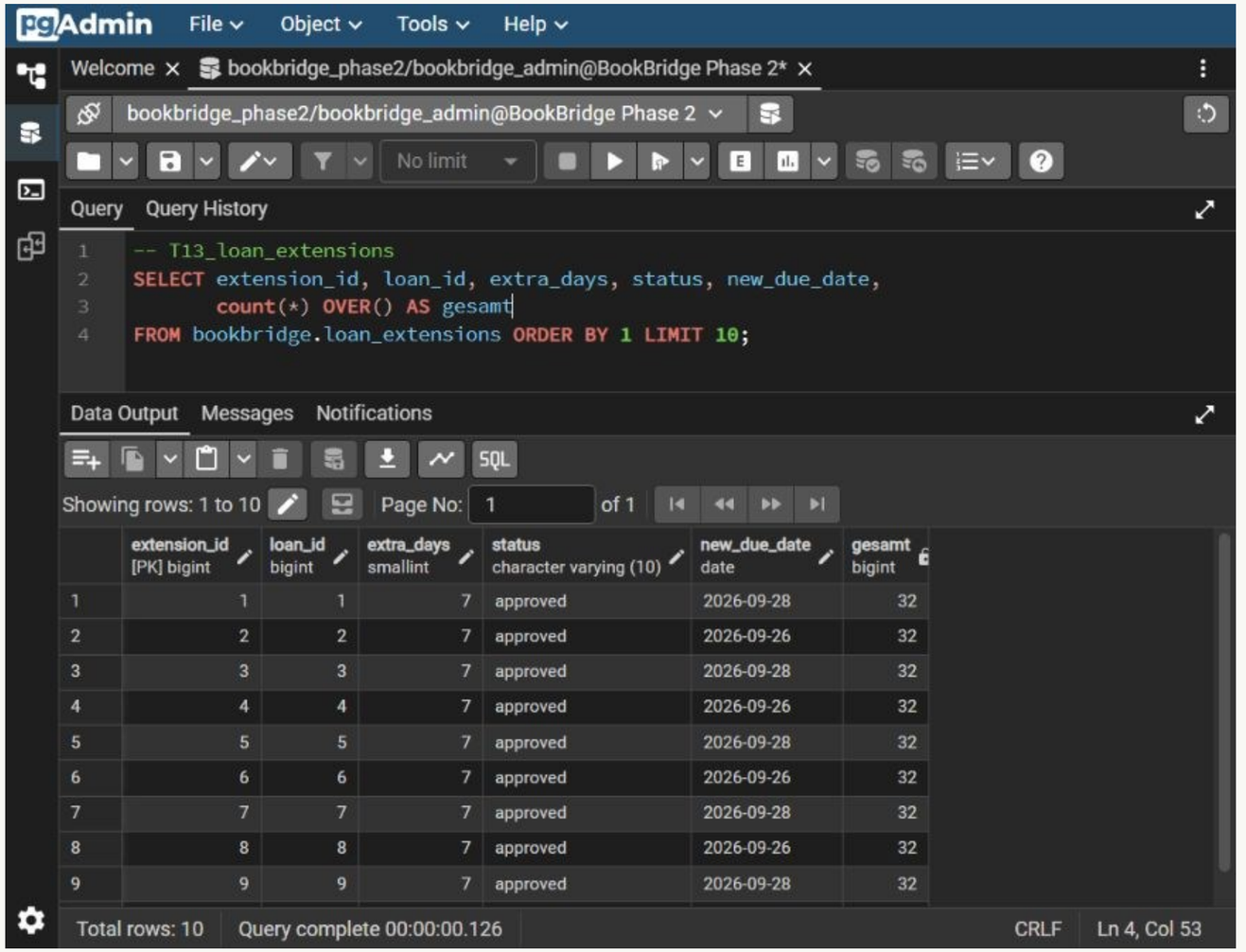
Beantragte und entschiedene Verlängerungen

CREATE TABLE / SQL-Auszug

```
CREATE TABLE bookbridge.loan_extensions (
    extension_id BIGINT GENERATED BY DEFAULT
  AS IDENTITY NOT NULL,
    loan_id BIGINT NOT NULL,
    extra_days SMALLINT NOT NULL,
    status VARCHAR(10) NOT NULL DEFAULT
  'pending',
    -- Weitere Spalten im SQL-Skript.
    CONSTRAINT loan_extensions_pk PRIMARY
  KEY (extension_id)
);
```

Jede genehmigte Verlängerung verändert die Frist genau einmal. Die eingefrorene Gesamthöchstdauer bleibt bindend.

9 Spalten / 32 Datensätze



Portozahlungen: payments

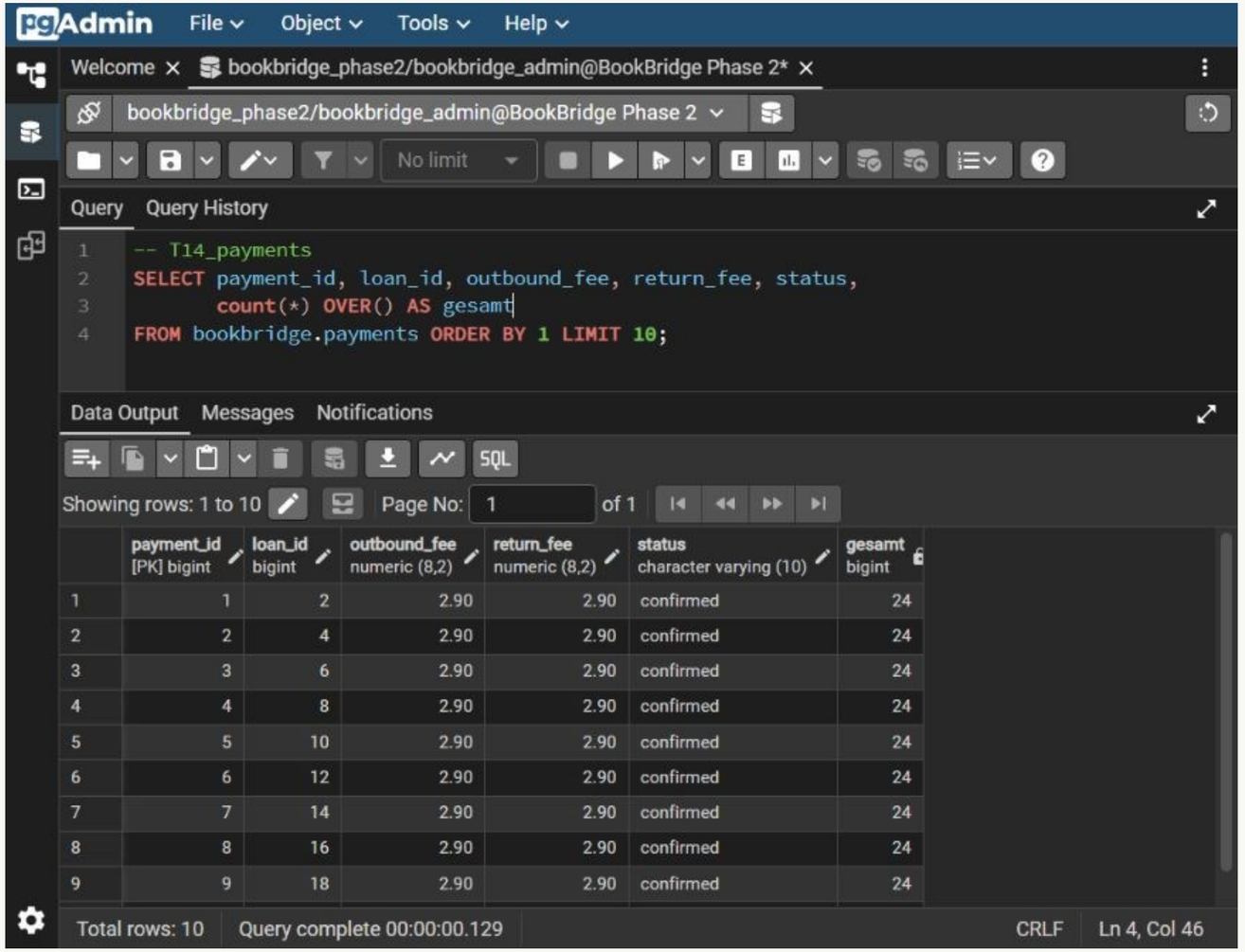
Portokosten und ihre Bestätigung

CREATE TABLE / SQL-Auszug

```
CREATE TABLE bookbridge.payments (
    payment_id BIGINT GENERATED BY DEFAULT
  AS IDENTITY NOT NULL,
    loan_id BIGINT NOT NULL,
    status VARCHAR(10) NOT NULL DEFAULT
  'pending',
    confirmed_at TIMESTAMPTZ,
    -- Weitere Spalten im SQL-Skript.
    CONSTRAINT payments_pk PRIMARY KEY
  (payment_id)
);
```

Eine Portoabstimmung je Ausleihe mit Versand. Nur bestätigtes Porto erlaubt den anschließenden Versand.

10 Spalten / 24 Datensätze



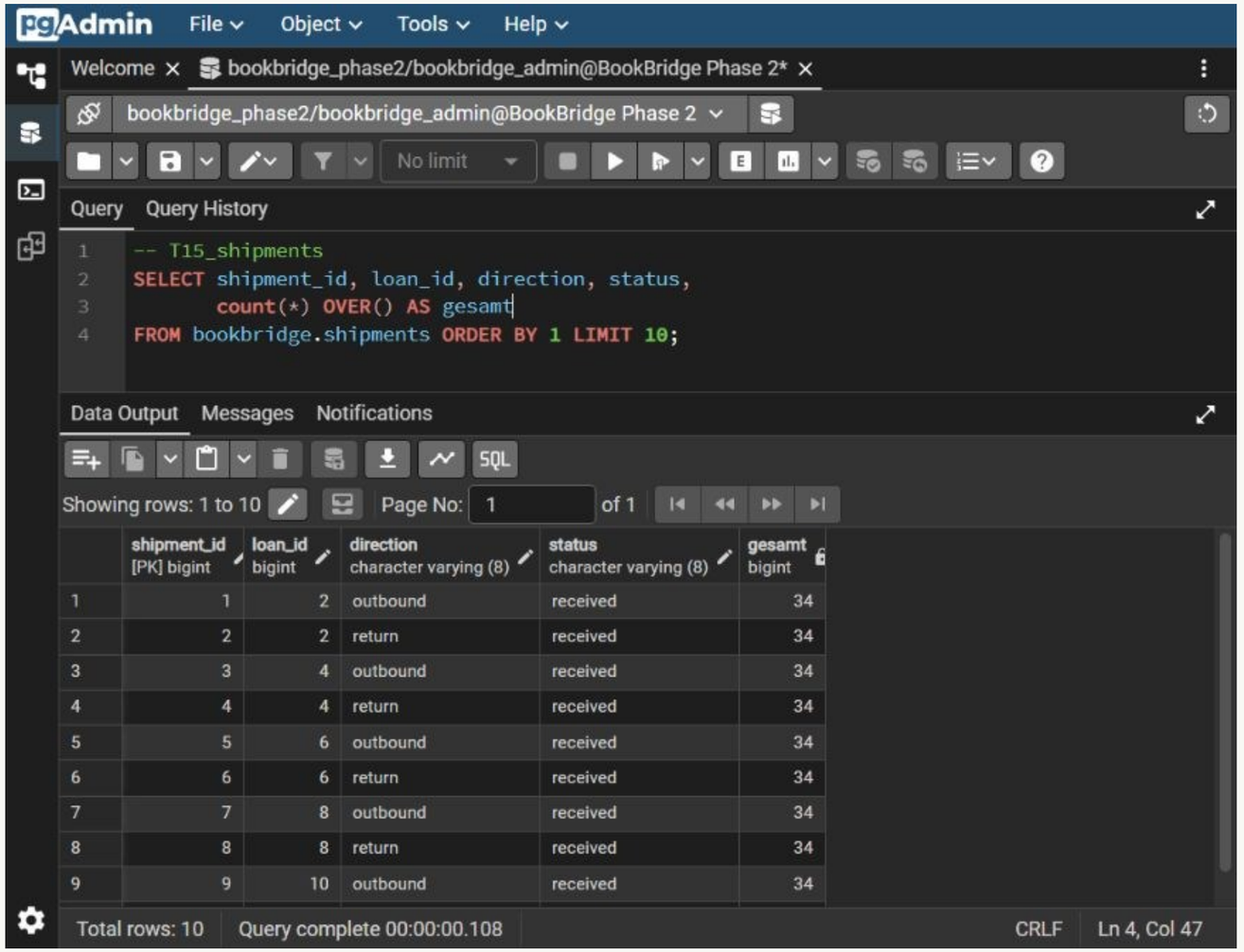
Sendungen: shipments

Hin- und Rücksendungen mit Anschriftshistorie

CREATE TABLE / SQL-Auszug

```
CREATE TABLE bookbridge.shipments (
    shipment_id BIGINT GENERATED BY DEFAULT
  AS IDENTITY NOT NULL,
    loan_id BIGINT NOT NULL,
    direction VARCHAR(8) NOT NULL,
    status VARCHAR(8) NOT NULL DEFAULT
  'prepared',
    -- Weitere Spalten im SQL-Skript.
    CONSTRAINT shipments_pk PRIMARY KEY
  (shipment_id)
);
```

Hin- und Rücksendung stehen getrennt. Eingefrorene Anschriften erhalten die Versandhistorie.



14 Spalten / 34 Datensätze

Benachrichtigungen: notifications

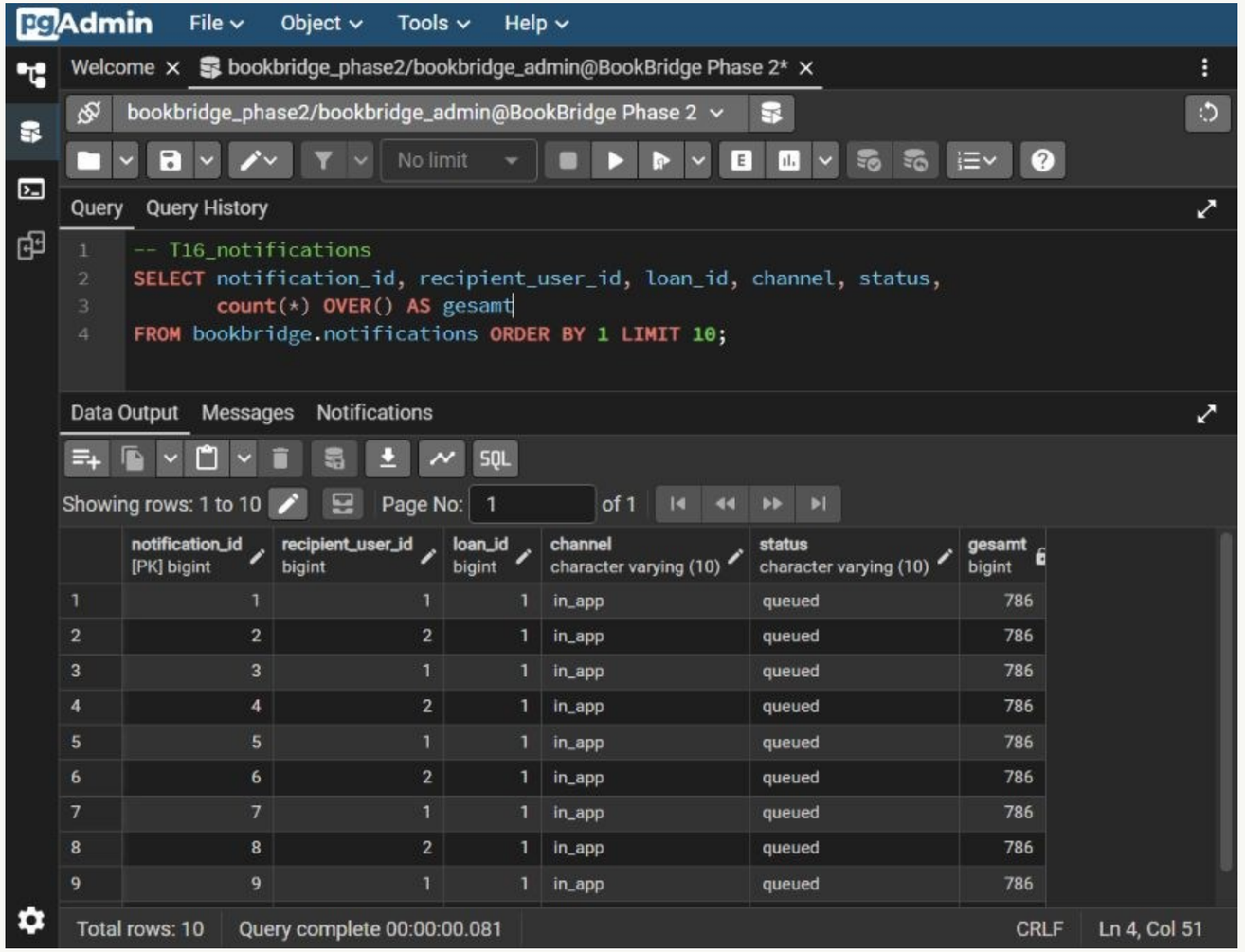
Private Nachrichten mit optionalem Ausleihbezug

CREATE TABLE / SQL-Auszug

```
CREATE TABLE bookbridge.notifications (
    notification_id BIGINT GENERATED BY
  DEFAULT AS IDENTITY NOT NULL,
    recipient_user_id BIGINT NOT NULL,
    loan_id BIGINT,
    channel VARCHAR(10) NOT NULL DEFAULT
  'in_app',
    -- Weitere Spalten im SQL-Skript.
    CONSTRAINT notifications_pk PRIMARY KEY
  (notification_id)
);
```

Nur Beteiligte erhalten vorgangsbezogene Nachrichten. loan_id kann für allgemeine Nachrichten fehlen.

9 Spalten / 786 Datensätze



Dokumentnachweise: generated_documents

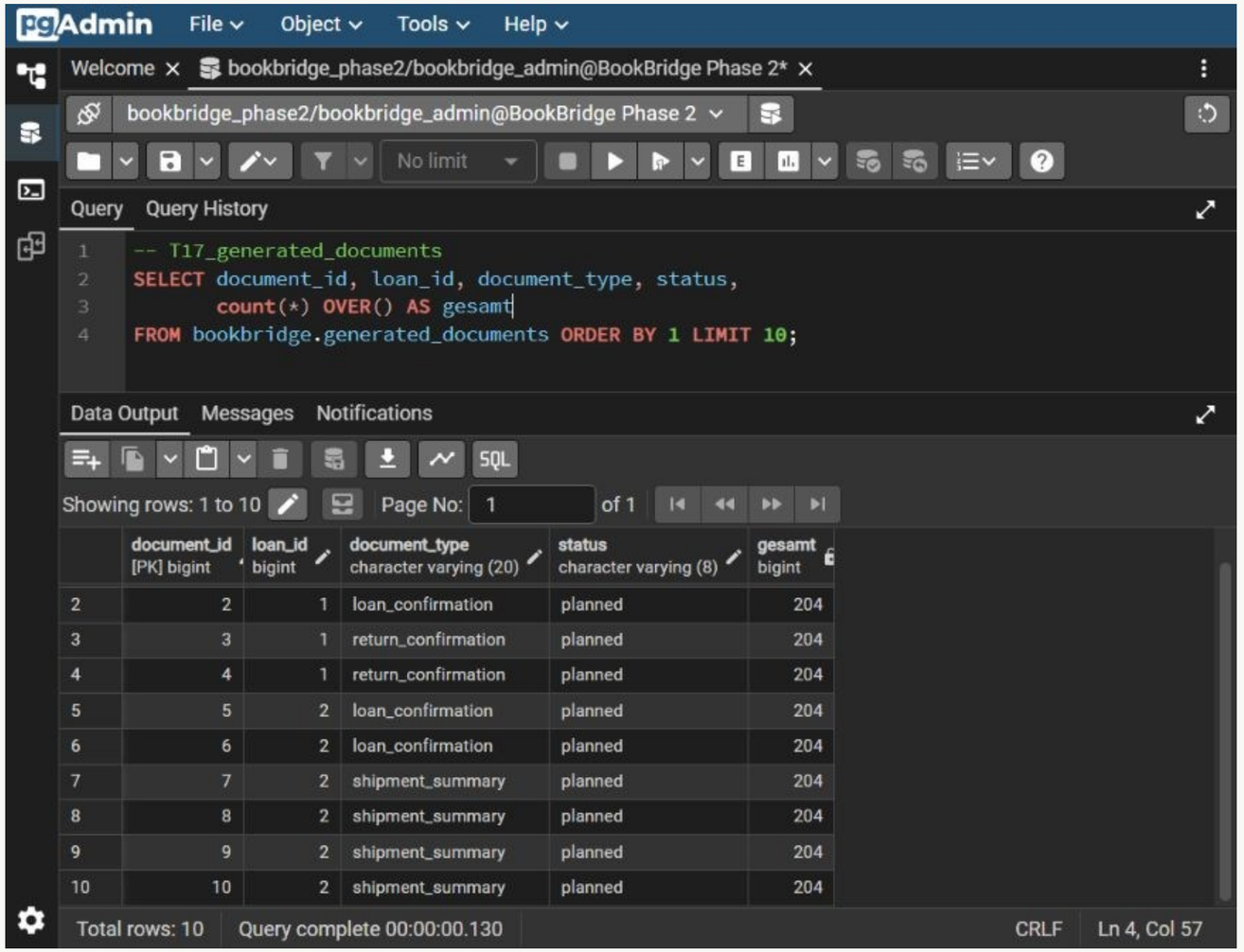
Nachweise als Metadaten und Dateiverweise

CREATE TABLE / SQL-Auszug

```
CREATE TABLE bookbridge.generated_documents
  (
    document_id BIGINT GENERATED BY DEFAULT
  AS IDENTITY NOT NULL,
    loan_id BIGINT NOT NULL,
    recipient_user_id BIGINT NOT NULL,
    document_type VARCHAR(20) NOT NULL,
    -- Weitere Spalten im SQL-Skript.
    CONSTRAINT generated_documents_pk
  PRIMARY KEY (document_id)
);
```

Empfänger, Vorgang und gegebenenfalls Sendung müssen zusammenpassen. Der Datensatz enthält Dokumentmetadaten.

9 Spalten / 204 Datensätze



Bewertungen: reviews

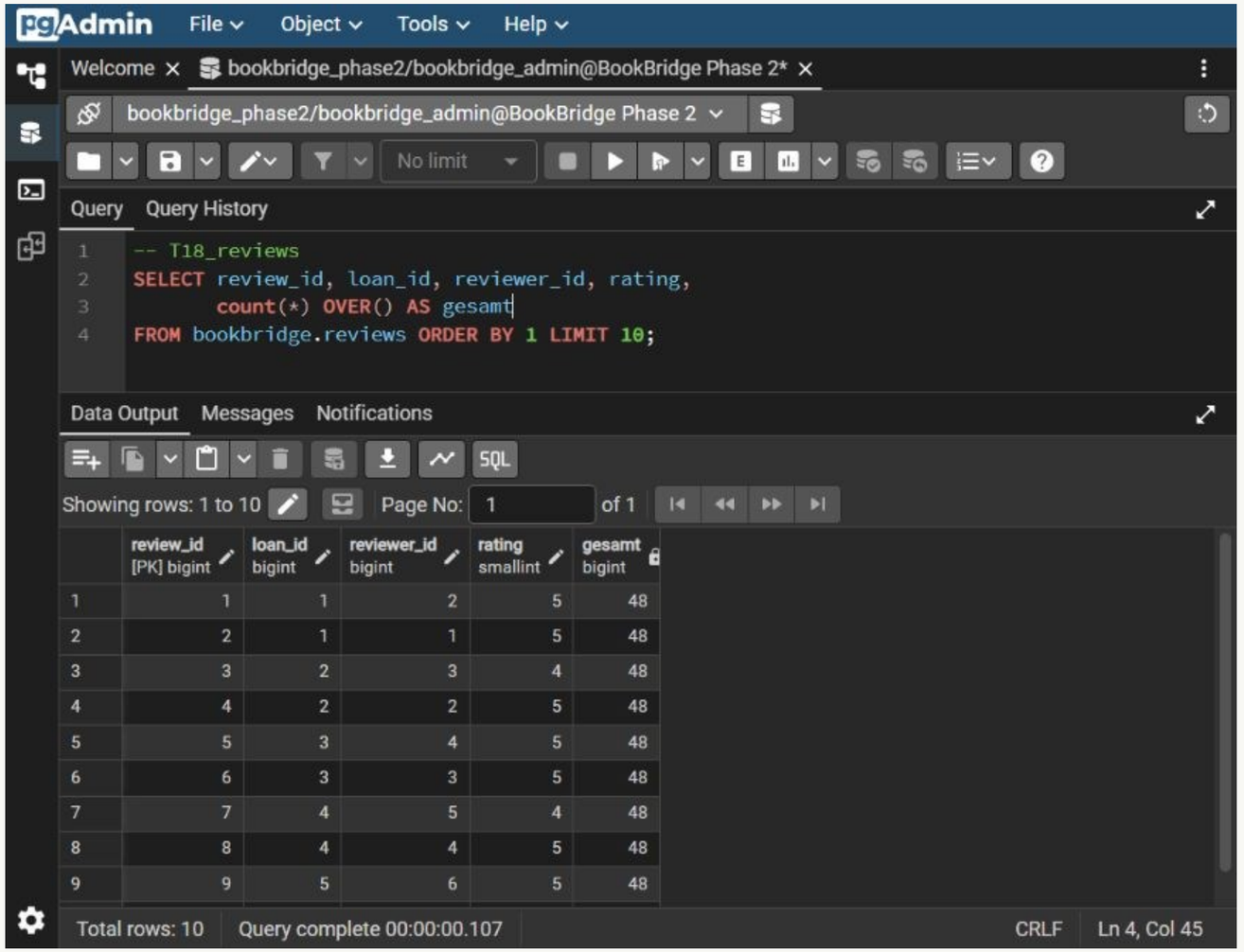
Bewertungen abgeschlossener Ausleihen

CREATE TABLE / SQL-Auszug

```
CREATE TABLE bookbridge.reviews (
    review_id BIGINT GENERATED BY DEFAULT AS
  IDENTITY NOT NULL,
    loan_id BIGINT NOT NULL,
    reviewer_id BIGINT NOT NULL,
    rating SMALLINT NOT NULL,
    -- Weitere Spalten im SQL-Skript.
    CONSTRAINT reviews_pk PRIMARY KEY
  (review_id)
);
```

Eine abgeschlossene Ausleihe erlaubt höchstens eine Bewertung je beteiligter Person. Die Bewertung liegt zwischen 1 und 5.

6 Spalten / 48 Datensätze



JOIN 1: Ausgabe, Exemplar und Eigentümer

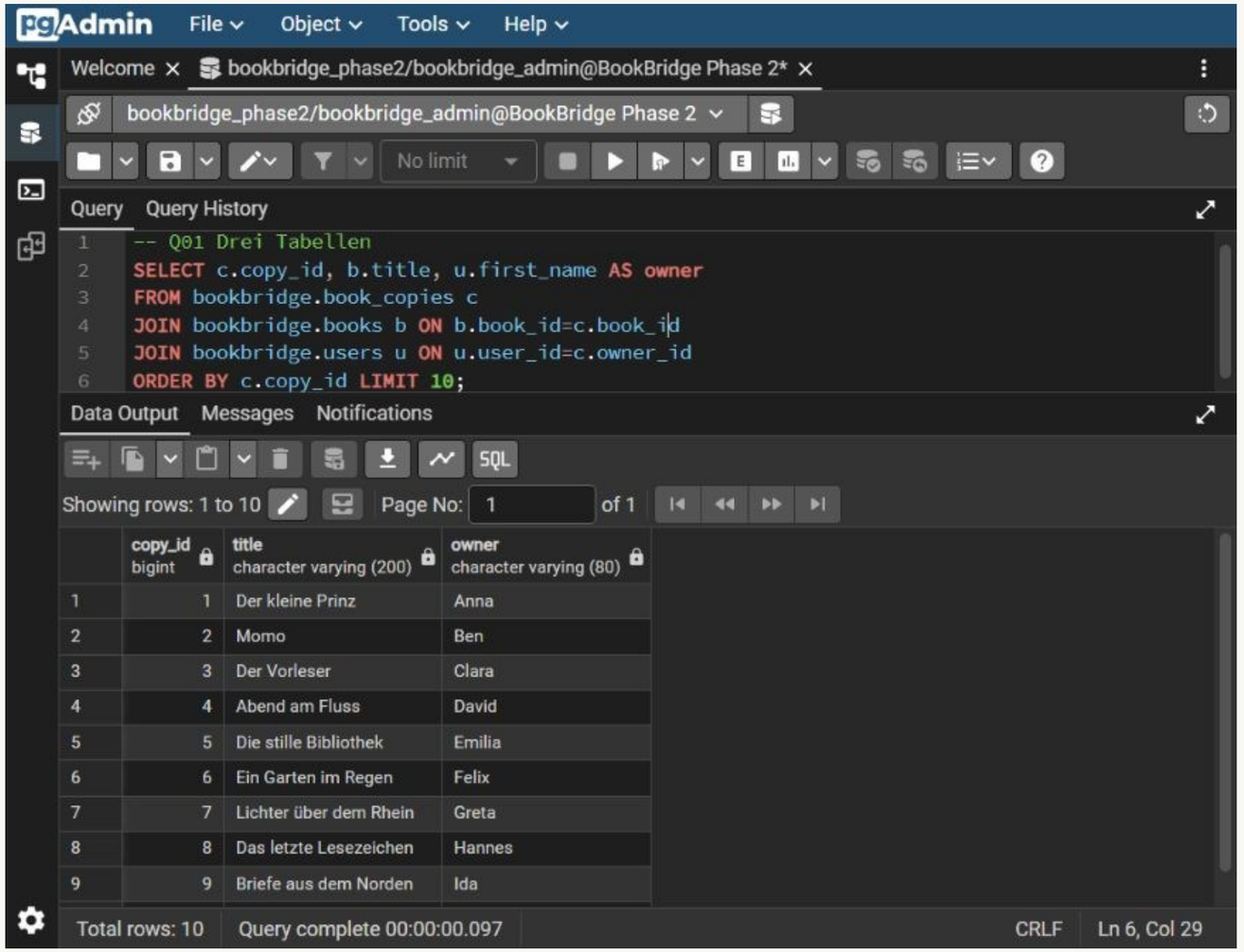
book_copies, books und users

SQL der Bildschirmaufnahme

```
SELECT c.copy_id, b.title, u.first_name AS
  owner
FROM bookbridge.book_copies c
JOIN bookbridge.books b ON
  b.book_id=c.book_id
JOIN bookbridge.users u ON
  u.user_id=c.owner_id
ORDER BY c.copy_id LIMIT 10;
```

Die Abfrage zeigt zu jedem dargestellten Exemplar den Titel und die besitzende Person.

10 von 24 Ergebniszeilen



JOIN 2: Ausleihe und ausleihende Person

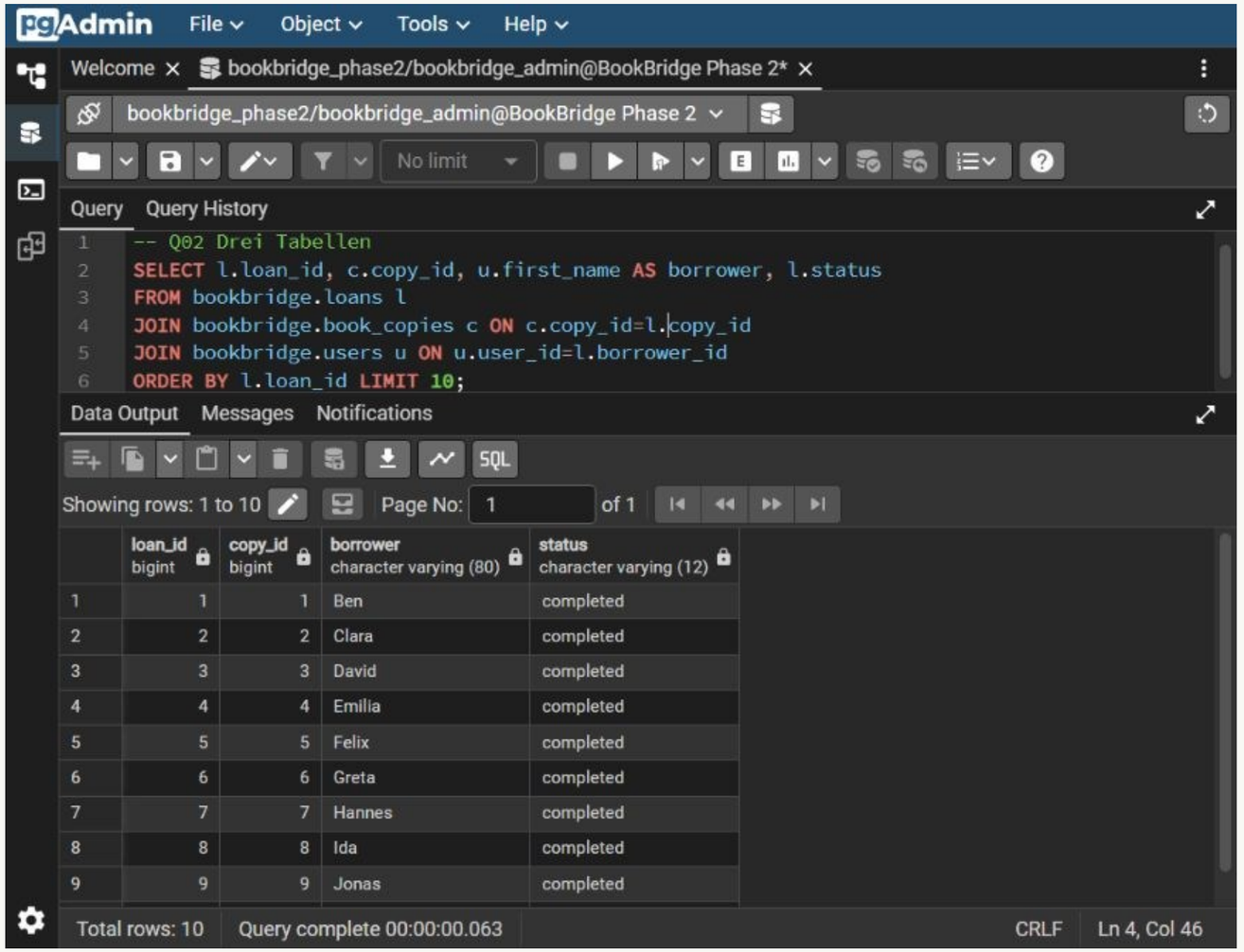
loans, book_copies und users

SQL der Bildschirmaufnahme

```
SELECT l.loan_id, c.copy_id, u.first_name AS
  borrower, l.status
FROM bookbridge.loans l
JOIN bookbridge.book_copies c ON
  c.copy_id=l.copy_id
JOIN bookbridge.users u ON
  u.user_id=l.borrower_id
ORDER BY l.loan_id LIMIT 10;
```

Die Abfrage verbindet den Vorgang mit dem physischen Exemplar und der ausleihenden Person.

10 von 48 Ergebniszeilen

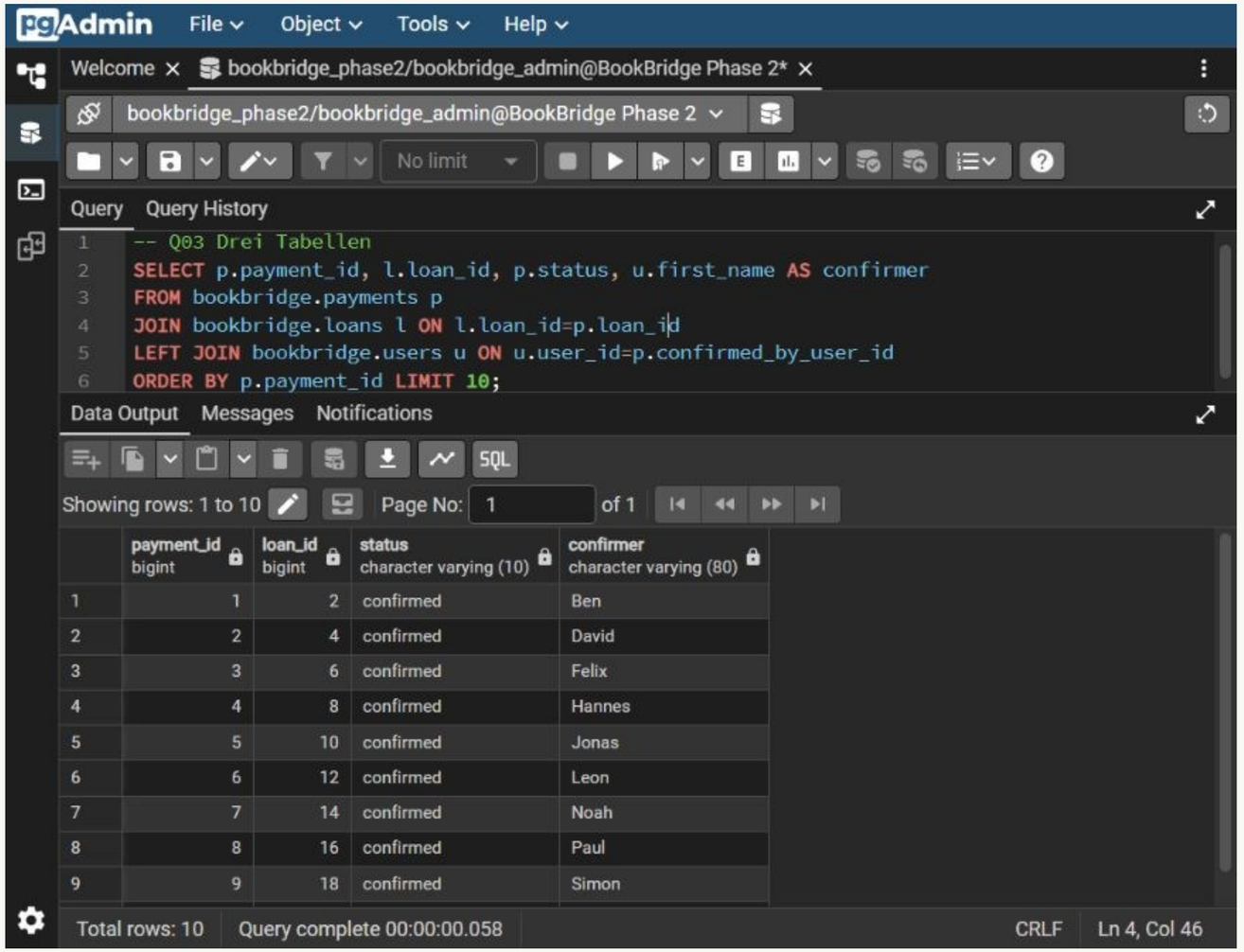


JOIN 3: Porto, Ausleihe und Bestätigung

payments, loans und users

SQL der Bildschirmaufnahme

```
SELECT p.payment_id, l.loan_id, p.status,
  u.first_name AS confirmer
FROM bookbridge.payments p
JOIN bookbridge.loans l ON
  l.loan_id=p.loan_id
LEFT JOIN bookbridge.users u ON
  u.user_id=p.confirmed_by_user_id
ORDER BY p.payment_id LIMIT 10;
```



Der LEFT JOIN erhält Portoabstimmungen, bei denen noch keine bestätigende Person feststeht.

10 von 24 Ergebniszeilen

Fachliche Integrität

Die Datenbank prüft einfache Werte und zusammenhängende Zustände.

Regel	Umsetzung
Identität und Verweise	Primärschlüssel, 32 Fremdschlüssel, NOT NULL
Wertebereiche	CHECK für Status, Dauer, Beträge, Koordinaten und ISBN
Eine bindende Ausleihe	Partieller UNIQUE-Index auf copy_id
Mindestens ein Autor	Aufgeschobener Constraint-Trigger
Übergabe und Frist	Workflow-Funktionen und tabellenübergreifende Validierung
Historie	Unveränderliche Teilnehmer und eingefrorene Anschriften

Einzelne erfolgreiche Schreibbefehle reichen nicht aus. Der Endzustand muss zusammenpassen.

Parallele Annahme desselben Exemplars

Ein echter Test nutzt zwei gleichzeitige Datenbankverbindungen.

Verbindung	Ergebnis der Annahme	Endstatus der Anfrage
Verbindung 1	Abgewiesen (55000)	requested
Verbindung 2	COMMIT erfolgreich	accepted

Ergebnis: genau eine verbindliche Ausleihe

Der abgewiesene Vorgang bleibt eine Anfrage. Die Datenbank verhindert die doppelte Vergabe auch bei gleichzeitigem Zugriff.

Integrationstests und Grenzfälle

23 von 23 Tests erfolgreich. Die Tests prüfen die Regeln R01 bis R20.

Prüfbereich	Reale Testfälle
Struktur und Abfragen	18 Tabellen, mindestens zehn Zeilen, drei JOIN-Ergebnisse
Anfrage und Verhandlung	Eigenleihe, Gegenvorschlag, Rechte, eindeutige Annahme
Frist und Rückgabe	Verlängerung, verspätete Rückgabe, beide Zeitumstellungen
Porto und Versand	Bestätigung vor Versand, Hin- und Rückweg, Empfangsperson
Schutz und Historie	Private Inhalte, unveränderliche Vorgänge, Deaktivierung
Transaktionen und Rechte	Rollback-Fingerabdrücke, zwei Verbindungen, Rollenprüfung

Die normalen Tests rollen ihre Testdaten zurück und vergleichen danach alle Seed-Zeilen.

Datenbankrollen und öffentliche Lesesicht

Öffentliche Katalogdaten stehen getrennt von privaten Vorgangsdaten.

Rolle	Erlaubter Zugriff
bookbridge_reader	SELECT auf public_catalog
bookbridge_backend	Katalog lesen und freigegebene Workflow-Funktionen aufrufen
PUBLIC	Kein Zugriff auf private Tabellen, Sequenzen oder Funktionen
Datenbankeigentümer	Installation, Administration und kontrollierte Seed-Erstellung

public_catalog enthält keine E-Mail-Adressen, Straßen, genauen Koordinaten oder privaten Dokumentpfade.

Die Anwendung muss ihre Benutzer weiterhin authentifizieren und actor_id zuverlässig setzen.

Installation und Reproduktion

Die SQL-Dateien lassen sich in einer leeren Projekt-Datenbank erneut ausführen.

Datei	Aufgabe
01_schema.sql	Schema, Tabellen, lokale Constraints und Indizes
02_integrity.sql	Trigger und tabellenübergreifende Konsistenz
03_workflows.sql	Fachliche Aktionen als Transaktionen
04_seed.sql	Sinnvolle Testdaten und vollständige Abläufe
06_access.sql	Gruppenrollen und öffentliche Katalogsicht
05_queries.sql	Ergebnisabfragen nach erfolgreicher Installation

Skriptreihenfolge der Installation: 01, 02, 03, 04, 06. Anschließend folgen Abfragen und Tests.

Reflexion: Modell und Konsistenz

Bewertung der fachlichen Entscheidungen

Die Umsetzung bestätigt die zentrale Trennung zwischen bibliografischer Ausgabe und physischem Exemplar. Dadurch speichern mehrere Angebote gemeinsame Buchdaten nur einmal, während Eigentum, Zustand und Leihdauer am Exemplar bleiben. Die Ausleihe bildet einen eigenständigen Vorgang mit nachvollziehbaren Teilnehmern und Ereignissen. Diese Entscheidung erleichtert spätere Bewertungen und die Auswertung abgeschlossener Vorgänge.

Die anspruchsvollsten Regeln betreffen mehrere Tabellen. Eine einzelne Spaltenprüfung kann weder die erforderliche Autorenzuordnung noch einen konsistenten Versandablauf absichern. Deshalb ergänzen aufgeschobene Constraint-Trigger die lokalen Constraints. Die Prüfungen beurteilen den fachlichen Zustand am Transaktionsende. Während einer zulässigen mehrstufigen Änderung dürfen vorübergehende Zwischenzustände entstehen.

Diese Absicherung erhöht den Pflegeaufwand, weil neue Status oder Übergänge mehrere Prüfungen betreffen. Die automatisierten Regressionstests sind deshalb ein wesentlicher Teil der Umsetzung und müssen bei späteren fachlichen Änderungen gemeinsam mit den Funktionen angepasst werden.

Reflexion: Konkurrenz und Integration

Bewertung der Ausführung und der verbleibenden Anwendungsgrenzen

Die Ausführung hat außerdem gezeigt, dass Verfügbarkeit eine abgeleitete Eigenschaft ist. Mehrere Anfragen dürfen bestehen, doch nur eine verbindliche Annahme blockiert ein Exemplar. Der Test mit zwei gleichzeitigen Verbindungen prüft diese Entscheidung unter Konkurrenz. Eine Annahme endet erfolgreich, die andere wird abgewiesen.

Für die weitere Entwicklung bleibt eine klare Grenze: Die Datenbank liefert Fachfunktionen und eine öffentliche Lesesicht. Ein späteres Backend muss Anmeldung, Sitzungen und die Zuordnung der handelnden Person übernehmen. Dokumentdatensätze weisen auf Dateien hin, erzeugen aber noch keine realen Versandetiketten oder Briefe. E-Mail-Versand und postalische Dienstleistungen bleiben externe Integrationen. Auch die Suche nach geografischer Entfernung benötigt bei höherem Datenvolumen eine geeignete räumliche Erweiterung und eine passende Indexstrategie. Diese Grenzen ändern die umgesetzten Tabellen und geprüften Abläufe nicht, bestimmen jedoch den nächsten Integrationsschritt.

Ergebnis und prüfbare Unterlagen

Die Implementierung liegt mit reproduzierbaren Ergebnissen vor.

Unterlagen	Verzeichnis
Vollständige Implementierung	sql
Modell aus Phase 1	design
Integrationstests und unabhängiger Aufbau	tests und scripts
SQL-Ergebnisse und Prüfberichte	evidence
Echte pgAdmin-Aufnahmen	screenshots
Bearbeitbare Dokumentation und Präsentation	docs

18 Tabellen. 32 Fremdschlüssel. 23 bestandene Tests.